

好，大家晚上好，大家晚上好过了一个11了。不知道大家都胖了几斤，反正我怎么没事共享出来声音，大家能听到我声音，能听到我声音。然后这个屏幕主要这个视频大家能不能听到，能的话帮我打个一呗。
Good morning, and welcome to that day.

Thanks for being here in sam cisco, where we are, and itt. 在线同学谁能听到帮我打一个屏幕的声音，好像能听到是吧？A long way. Since then, back in twenty twenty three, we had two million weekly developers.

好，行，没问题没问题，没问题。好，那咱就直接来。这个十一其实发生的东西也挺多的。最这个叫什么？最吸引人的肯定还是咱们都能想象到，就是open I这个def day。然后def day发生之前，其实就在咱们放假之前，open I直接发了一个大招，大家知道吧？这个32 sora to这个模型，而且这次做的还比较有意思，是什么呢？他做了一个应用，他做了一个APP，我不知道大家有没有人用了这个solo 2，可以给大家待会儿到了待会儿到了给大家看一下。

讲这部分好，然后当然这前两天其实最大的就是这个death day这个开发者日，每年一年一度的open I春晚就是这个东西。然后他开始今年开始这个屏幕也很好玩，大家可以看看这个就是一个贪吃蛇，然后不断的吃掉那个树，最后实现这个到0的时候，开始了这么一个东西，很好很有意思的一个东西。然后good morning，我们就开始讲几个东西。奥特曼说了这么几个东西，他在前年二三年、24年二三年、24年、25年的时候，其实核心看这25年，你看一下25年的时候他有多少开发者呢？他有400万个开发者，这啥意思呢？其实基本上已经可以成为是一个很大的操作系统生态。

你是不是没开共享屏幕？是吗？对我靠，没有。Sorry, 抱歉。

好像小通的共享屏幕怎么点不开了？我靠，点不开了，我退一下，我重新进一下，抱歉。大家稍等一下，小平老师调试一下设备。

当前共享窗口异常咋回事儿？不让我共享窗口，刚才还OK，稍等，好像出来了。这次大家看到了，能看到吗？可以的，没问题了OK抱歉。对，刚刚没没就不知道这个怎么怎么没共享出来。

行，然后就回到这儿，我们看他现在有400万个开发者，然后有之前给大家说过那个ChatGPT做了一次分析说全球的人到底在怎么用ChatGPT。然后有八这个800个million，就是8亿的周活用户，就是chat BTO8亿周活用户。这个其实对于一个互联网产品来说是一个特别可怕的一个量了，没问题，应该能听到了。现在对，特别可怕量。然后这个数你看现在大家知道像国内的这些大模型平台，比如像什么火山，像千万，这些其实都在说的一件事情就是我有多少开发者每天会消耗多少token这个量，对吧？然后我是第一次看到有一个open I定了一个新的单位，就是每分钟多少token，基本上大家搜像这个火山啥的都会说，我每天会有多少万亿的token消耗的量，还是多少，反正就是非常这个挺多的。第一次看到这个open I按照分钟来说，那我们看看这到底是多少分钟，这个到底多少量？六个billion其实就是一般就按比例来参数来去搞了。

那一分钟那就是60分钟乘以24再乘以6，那其实就是8640个billion，也就是接近一个T1天接近一个T的token，这个放的其实放在国内来说的话，其实并没有那么可怕。你看看，我看能不能找到火山，火山引擎每天的这个token量，我记得好像之前看到了一个数。你看5月25年5月底的话，每天的tok量应该是16万亿，16万亿就是一万多币。对，一万多币的这个token，比起他这个还是多的。所以但是大家可以想象火山确实是一个极大的用户群体，所以这个open I的这个量，我觉得也是媲美一个大的云厂商的这量还是非常大好。

他就开始说了，就是奥特曼就开始说这次发布的几个东西，他弄了一个名单，这个名单是啥意思呢？是每个有这些开发者全是开发者的名字。大家可以看自己看这些开发者其实有很多的中文的名字，其实就是华人的这个名字。

一绿色的是花过了一t token，然后蓝色的花过100B的，然后10B的这个token的消耗量的一个开发者全给扔上去了。而且都是现场的开发者，所以把这个东西当做一个非常重要，这跟国内差不多，就是大家都现在都拿这个模型的token消耗量当做一个特别大的卖点来去说话。我们看看它底下，我看看哪去了。对他四个主题，就这么四个字主题，一个叫apps inside chat PT，就是在chat PT里面的APP。第二个叫building agent，怎么构建一个agent。第三个是写代码对吧？大家可以想象就是它的code dex。第四个有一些API的更新。

就这四个大主题我觉得其实最重要的其实就是第一个，或者说对我我觉得基本上可以说是第一个是最重要的这样的一个主题。对，大家有问题就随时说，别到时候我又说了半天，没看到屏幕上，我们就看第一个主题，第一个主题是这个acts in ChatGPT。对，然后这个玩意儿是啥呢？就看他这篇博客，就是ChatGPT里面的APP。大家想其实比较容易想象到就是chat BT里面的GPTS对吧？这个我不知道大家以前有没有用过，GPT s就是上一波chat BT尝试想去做应用，做这个开发者生态的这么一个产品，那本质上是啥呢？你看直接在这儿创建一个agent，然后我就说就行了。比如说我想创建一个辅助人们买买家居用品。

A站，对，然后他就会帮我去设计说到底应该怎么定义这个agent，这个agent的它的这个描述怎么描述这样的一些东西，他就会给你写一大堆这个agent本身的描述，就干这事儿。所以其实背后还给我起个名字，行，对，然后他就给我们看已经写了一大堆了。一大堆是啥呢？

这个东西的描述，这个东西的instruction，就是它的定义，就是这玩意儿到底是啥，对吧？就是这个agent是干啥用的？然后写了一些开始的比较合适的绘画的开头，然后底下就一大堆，你可以上传你的知识库。比如说咱们这个直播的群里面有一些同学有一大堆自己的知识，有一大堆自己的经验，然后写成了word，然后就可以传进去。他会建议还可以配一些工具，比如说搜索画图，然后写代码之类的，这些其实都可以去构建，然后就出来这么个东西。

这个就是它上一代这玩意儿就是所谓的一个GPT s对吧？这个GPT很多GPT对吧，就是GPT s这就是他上一代想去创建应用的这样的一个产品。那出了这么个东西，看起来也像那么回事，但是其实使用率很低。而且问题是他其实没法给这些产品形成一个很好的一个闭环，一个unitization就是盈利化的一个闭环。所以这个产品其实这个也就那么回事儿，这个是半死不活的这么一个东西。这次这个chat BT又想搞一个新东西了，那我不能叫GPT s了，我就叫APP那apps in chat BT这么一个东西。然后大家如果上网冲浪很早的话，还会记得他很早的时候，最早的想尝试做这件事儿的时候，做的是叫plug ins，就在GPDS之前叫plug ins，也是类似的东西。

其实他希望开发者把自己的应用或者应用的这个方把自己的应用里面的API接到它的这个agent里面，可以叫当时叫agent里面，然后形成一个生态也失败了。然后GPDS可以说也是也算是失败了，所以这次他又想尝试做一个新的。这次做的东西我觉得就比这个GPT s当然比GPT s肯定是进步了很大一块了，进步了啥呢？就是他能不能成功还是不好说的，那我们可以看，但是可以看出它整个在他构建这个生态，构建这个用户体验的这个地方，走了一很大一步。

你看他的这个介绍片其实很就很直接了。你看bookings，对吧？然后用哪bookings呢？找一个什么东西fima，用cosa教你什么东西，你可以直接去聊聊，跟APP去聊天儿。比如他第一个例子就是我想创建一个party night的这个叫什么播放清单，然后可以先看一下这个大概是啥意思。它就会蹦出来一个说你是可以其实可以用spotify，大家知道这spotify就跟QQ音乐什么的障碍。

网易云是这样一个东西，他就会去调这些第三方的应用来去干这事儿。这个很容易想象，但是它掉出来这个东西是一个像应用的这么一个卡片。可以说然后这个卡片其实里面还能操作，还能做一些互动的行为，就可以直接创建出来了。

Zillo是类似于贝壳这样一个东西，然后他就说再看看萨斯这个城，给我看一些在在的房子有哪些，咋他他就出来这么一个东西。可以看到其实整个的这个地图和地图里面的这些信息都是在当前的这个chat列表内的。其实他就是他没有推，没有离开这个APP，你可以一直在这个APP里面操作，然后看信息，然后继续去问对吧？你可以说这个在什么公园的行走的距离范围内，然后有一个什么什么完工的basement，这样的一些东西，就这么个东西。所以这玩意它就是它的这个下一次就最新的一次想去再在他的ChatGPT里面去构建应用体系的这么一个算是野心又一次尝试。

那我们看他做了啥呢？可以看看。对然后刚才那套东西肯定是要结合开发者去做的。就是它不是一个单纯的open I来去做的事情。所以他要结合大量的开发者一起来去构建现在这一代的应用。所以这个链接的方式，对你看所以它要构建什么呢？

所谓的叫APP SDK，它又推出一个新的SDK。大家知道听听这节课的同学知道这个，我看下有一大堆的SDK，什么agents SDK，然后这个乱七八糟各种SDK都有，都是一大堆。然后他现在又推出了一个新的叫APP SDK。既然叫SDK，那其实目的就是为了嵌入到其他的APP里，对吧？所以他这次的目的就是我不光想去直接植入你的这些底层的API，我还想直接嵌入到你这些APP里。当然这嵌入的APP的这个形式，他们还在这个我估计还在快速的改变中，他这个肯定不是一个最终态。

那它这个核心点其实就是他想把所有的应用融入到这个自然对话中。就像刚才说的，我不用去跳转到其他的APP里，不用去改变用户的这个交互的界面。我直接在ChatApp里面边聊边用，然后边去操作，直到最后完成用户想干的事情就可以搞定了。所以他现在拿这个东西非常大的一个叫什么好呢？

这个叫这叫什么吸引人的这个点，其实就是刚才最开始给大家展示的chat BT现在有800个million，8亿的周活用户数。好，大家其实都想要这8亿周活用户的流量，对吧？所以他现在就拿这个东西来去当诱饵，来去吸引所有的应用接入到它的这个接入被接入到ChatGPT里面。可以说是对。所以它有结合了一大堆，基本上所有的用户从免费的tier到pro全都有所谓的它叫什么应用apps in chat BT这个功能，你可以看到像bookings，就订飞机酒店的canvas对吧？就是画图的、画画海报的、考试、学习的f gma什么这些东西都可以干点事情。

然后对我们其实简单的看一下这个直接我拿举个例子，直接我这个chat PT应该已经升级了。比如说我们就拿他的，你看我就拿他那官方那个例子来去举例子，你看直接写cosa然后我想选点。大模型相关的课程。

对，然后大家有啥问题，可以随时发我对，你看他就会开始干嘛呢？他这个时候判断他要去找他有哪些应用了，对吧？然后他就去开始去尝试去对接，这时候他跟以前的所有还有最大区别。你看这它就蹦出一个区域，这区域是干嘛？这个区域其实如果大家想看这个技术上的话，直接我这儿点开它就这个有东东懂技术，大家一看就大概能明白这是个什么东西了。这个是一个让我inspect一下这儿。在网上。你哪儿去了？

不是这个，这。这是一个i friend，但是怎么找不出来了？算了再说。对，这就这个中间其实是一个其他网站的网页。对，然后我现在去点说我要去连接用户，因为它等于是open I的方式来去连接coursera。

对，就是在ChatGPT里面去连接我在cosa上的一个用户。这样的话他不就知道说我到底是谁了。然后他就可以以我的身份去可以去找资源去调这个视频出来这样一个事儿。所以你看他等于说我只要授权了，那现在chat BT其实就可以以我的身份去克算上去找课。

出来了一个deep learning AI的课，对吧？我觉得OK，那我就播呗。In the last video you explored, some of the computer are increasing the size of the dataset to train your model on increasing the number of parameters.

我把一播的话，你发现这个视频就会浮到表面，就跟这个所以这玩意儿是在他的这个描述里面叫PIP，看怎么样，好像不在这里面，他叫画中画。PIP这个听的不多，就是picture in picture，就是大电视上那种画中画。然后他在拆这个里面也做了很多东西，你看这个我随时可以去聊，然后我还可以随时问，说刚才他讲的是ut Operations对吧？这样的话我就他就不会干扰，不会因为聊天去干扰到我看这个视频的一部分，你看他刚刚讲的是啥Operations，讲的是大模型的预训练什么这些东西对我就可以继续去问，随时问。所以就这么个东西。然后这个东西直到我觉得我不想看到了，然后这个应用就结束了。

所以这个看起来是什么呢？就是我的问题可以直接调用一个外部的应用，外部的应用cos 2这块其实并不是那个APP，这就是它的网站cosine这家公司的网站来去产生的东西。我只要点开大家其实大家会看到，我其实跳的是cosa的官方这个网站可以直接去去点，但是我没有找到直接跳到这个应应该是这个。对，对，直接跳到这个应用他的这个课程的页面就OK了。对，所以就是这样一个东西，然后你可以随时在里面去跟他聊，然后问他问题，然后结束的话就OK了。

如果你想继续去让别的应用去做点什么事儿，比如说这个找看吧吧看吧，帮我画一个把刚才学课程内容的课程海报。然后他就会去继续去调这些APP去干。他其实跟传统，跟上一代的，我们说这个也不叫上一代。其实就是在他出现之前的，我们现在怎么去做这件事儿呢？那无非就是去我把这些canvas，把这些coursera这些在线服务它的信息，把它的数据全都暴露成API，暴露成MCP对吧？然后我就直接按我的意图让模型去选择性去调，到底调哪一个APP的什么功能这件事儿。

为什么这件事其实做不好呢？或者说体验不好呢？大家从这个过程就可以看到了，就是呃呃有两方面。一个方面就是说我如果我现在如果我直接说我想学点大模型相关的课程，好，那现在到底会出来啥？

比如说我们直接去这个，比如从deep sick，我们就看看deep thick，你直接输的话，他肯定会出来一大堆文字性的内容，顶多给你点链接说那我你可以去这儿学，去那儿学，然后给你一些链接，你直接跳过去就好了。但是你其实要的是直接去干这件事，你想的是我直接想学。所以在这个时候其实就给你一个很好的机会，是我是不是可以把所有的过程全都在我的这个chat平台里完成。

至少说这个chat BT这次新出的这个open I新出的这个叫APP in ChatGPT这件事儿是可以实现这件事，可以实现这个功能的，而且整个的用户体验非常的一致。你看这个他就开始给你生成了几个cma。这个cma其实就是刚才他会结合上下上面说的这个cosa的课给我生成的。这个海报不是很好看，但是这个就还行，就这么个东西。然后你甚至可以去调别的，就可以随时可以把这些APP给换起来，而且可以直接存在这个chat BT这个应用界面内去完成这些行为。所以这个就是它的第一个点，然后这个叫x in chat VT整个底层的技术还是基于MCP的所以他这一点做的很好，就是他没有自己去构建一个再弄一个新的协议出来。而是现在大家的MCP都已经很很很行业标准了，很成熟了，所以可以直接干你搞定这件事儿。然后再晚一些，他会去接受所谓的应用提交。

说白了就是真的开始尝试构建一个f store了。但是这个APP store大家可以看出来最大的区别就是啥呢？就是它跟苹果最大的区别是什么呢？就是苹果它构建的是一个应用的运行平台，而用户的内容全都是应用它上面的APP给提供的。

而chat BT其实不单纯，他后面会有一个说法就是chat p其实做了非常多的别的事情，然后那你应用你就每个应用就应该提供用户需要的那一点点小内容就行了。他不需要你去去构建整个的所谓我们传统意义上的这些应用的内容。对，所以他你看spotify make a play list什么的，这些就可以直接实现。然后可以给大家看一下这个在手机端的样子，我看能不能打开这个东西。还是按之前那个说法，我录一个手机的屏。

然后我们比如说还是创建一个这个。帮我搞着。我去。

哎。

传过去。

About five. 你看它只要是输入这个应用，就直接搞出来这个东西了。但我觉得这其实有风险，你想这些他以后肯定一大堆广告在这争这个位置帮我。我去他现在这个中文输入有问题。Make a list for my. Party tonight.

不知道为啥，这个总是说有问题OK。对，然后它的手机端的这个体验其实会比这个网页端还好一点。然后你看他现在会问这个specify去干一些事情，然后这块会弹出一个界面，这个界面其实就刚才的这个内容，就是你的。可以直接交互的一个内容。然后他现在说你看它生成的是一个list，所以可以是想象是什么呢？就是这个东西肯定是一个单独定义的一套前端框架，而不是那个spotty自己想做成啥样就是啥样的。这个有点慢，我们就等不等。正好我就给大家看一下这个，给大家看一下sora，大家有没有人用了这个sora的，现在咋了？

夏天。对，可以看到这个sora它现在做成一个APP，这它做成了一个APP。做成APP的最大问题是啥呢？就是他们号称最近这一周大家肯定也听过了各家商业评论都在干这事儿。就说这个chat b想搞自己的抖音了，对吧？但是你会发现啥，反正我自己访问的时候贼慢，就是几乎刷不出来这个内容，实在是太慢了。你看整个交互界面极其的抖音，他做了一个小特殊的功能，真的刷不出来。我看看我把它换成5G能不能看出来。

看不着我去。天哪。Sorry, 太慢了，现在是。算了，我们就先放在这儿，看看能不能露出来，待会儿可以看一下，反正就是内容好像有点意思。我如果说网速真的快一点的话，我觉得还是挺有意思的这么一个产品。但是可能确实是用户量实在太大了，所以这个直接就给搞崩了。

对，然后你可以看到这个东西。所以如果大家这个现有应用是专门做什么做出海的话，那我觉得其实这个APP in chat BT真的是可以非常值得关注的一个点。你看现在跟谁跟他合作bookings定基酒的canvas对吧？做这个海报的可sera学课程的，然后expired a也是这个顶级9的FIGMA做应用的。他这个挺有意思，我觉得这个叫直接你扔张图，就是大家经常说想到的一个点子，然后你能帮我把它变成一个时序图吗？或者这个the diagram直接变成你甚至？比如说如果以后他能接什么做应用那种rapper什么的，其实甚至都可以一键的让他把这个点子实现出来，做成一个应用对吧？其实都是可以做到的，然后像spotify就是播歌的，我的网不知道大家有没有有没有人用过，但是确实在chat BT的最近的这个网速实在是太慢。

好，然后像ZEALO就是看房地产的，我觉得其实这个是更有意思的。就是大家最近十一出去玩。那肯定会不断的用到地图这种应用。地图的应用很大的一个问题是啥？

就是他的筛选条件或者排序条件是一个完全由他来去决定的，你没法靠自然语言来去筛选。比如说帮我找一个这些点，比如说离公园五分钟车程以内的这样一个点，然后他能筛出来吗？其实现在完能不能做这事儿，完全取决于像这个这个高德，像百度地图、腾讯地图，他们能不能实现这个功能。但事实上，其实大家如果有有这个懂一点技术的话，肯定知道说我要实现这个东西，其实是我要不就是写一条像SQL语句这种查数据库的语

言，要不就写一些python写一些代码来去筛选一下就能筛出这些东西。当然效率是另一回事。所以其实如果是让大模型来去参与说筛选当前的，比如说帮我找这些点里面离烤肉店只有五分钟车程的这么一个地儿。这件事情理论上其实是可以靠大模型现生成CQL现生成代码去筛选出来的。而这件事情传统意义上都是要让这些应用公司自己的产品经理来去排序，来去决定到底做哪个，不做哪一个这样一些事情。

而这事儿其实你说合不合理呢？我觉得倒是不那么合理。就是以前，就是因为开发的这个带宽有限但如果现在能够把这个口放开，那又会有有边缘有长尾需求，那就让他先生成就好了。对，所以就是这个东西，这样的一些长这样一些APP先跟欧派I合作了，接着会有哪些呢？像什么uber、target这个trip、weather什么的，全都是即将跟他合作这些公司。所以他还开了开放了，就写了两个叫开发者该指导这样的一个东西就给APP的开发者应该咋搞，以及给APP的设计师应该咋搞。这两个东西开发者咋搞呢？这个东西我就就可能这个没啥意思，这个可能大家没有那么大兴趣。

然后它里面其实提了几点，这些前面些不太重要，核心是这个东西就是谁能够接入ChatGPT这件事情取决于他们是有一些标准的，就是不是所有的应用开发者只要搞出来就能立刻要能接入。它还有一些比较高的要求，比如说什么什么应用才能满足这些要求？叫他要qualify，要qualify啥？满足这些要求以后，满足什么要求，满足他这个design概览，就这个设计感了以后，才能够进入他的所谓叫增强分发强化分发机会的这么一个列表。就是你的应用必须得是啊他看的好的，或者说符合他的设计规范的，才能够享受到他的这个算是流量支持聊聊流量扶持这样的一个范围。

所以其实对于开发者来说，你如果只是把这个按照他的协议接入了，那也只能是能用。或者说这个用户如果想用的话，能用，但是他如果你想把它做得好的话，必须得符合他的这个design橄榄，就是设计橄榄。对，然后开发者这块儿，我就没啥说了，这块儿就对，我不知道咱们有没有同学是做出海应用的，有没有？有吗？有的话帮我在群里打一下。对，这个如果做出海应用的话，这个design概念还是很值得看的。就是这个设计概念和这个开发者，你看它是什么呢？就是这些东西刚才给大家看到了那个spotify，或者像这个conserve什么的，其实都是出来一个一个的像卡片似的玩意儿，对吧？就像这个东西，这个东西其实是个卡片，你看它其实是一个音浪的一个卡片。刚才这个可塞热克，其实也是一个inline的卡片。

或者大家简单理解成就是一个网页，但是这个网页是一个特殊设计的一个网页，必须得满足他的一些设计要求。比如说得足够简单，而且这个里面千言万语其实就一句话，一定得是你看一定得是ChatGPT的自然延伸。说白了就是你得是ChatGPT被被chat PT用的这么一个东西，而不能够在抢夺它的这个GTP的这样的一个地位。你看它的x GPT用来控制系统级别的元素，如这个语音、chrome样式，什么composer这些东西。而应用开发者用来去干嘛？

就来去定义里面的内容，来去给用户提供价值。所以这就是把应用的开发的边界给限定死了，你就只配做里面的内容输出，你不要去做那些更更复杂的这样的一些更全局的一些事情。所以他就是干这件事情，然后避免你去抢他的这个位置，所以inline就是每一个应用其实是一行。然后你看你可以去搜，然后你可以去展示这个图片，你可以去左右滑的这样一个展示都OK，但是也可以展示地图，但是你能只能在你的这个卡片里显示信息，你也不能放太多。

我不知道大家有没有互联网这个领域比较待的比较久同学，会感觉有点像啥。不知道大家有没有听说过一个词儿叫轻应用。轻应用。当年在国内的互联网和像苹果什么的里面也都火过一个。

当时我记得百度其实刚是最火的好事。十几年前了，百度他搞了一个轻应用，是啥呢？其实就是在百度的搜索裹里面，蹦一个内容出来，直接给你呈现这个要的东西。我们直接看大家如果用百度什么搜什么北京的天气。我看比如北京的天气，这玩意儿你看整个这个东西其实就是一个所谓的轻英勇。

我去现在搞的百度咋搞啥东西，搞得这么复杂庞杂的这种东西，这个东西其实就是一个轻应用，所以这个事儿也不是第一个第一天做了。当时做清运的时候，他还提了一大堆什么行业这个协议，就是行业的联盟，大家一起做这个轻应用，然后都基于这套协议来去构建这个小的应用，但事实上后来证明，其实这一类的应用谁做起来？也就像这个小程序这种东西还算是比较有用。而百度这种的清运，其实就用的很少。

后来苹果也搞了这个东西，苹果这个东西，它其实主要是这种，就是嘎蹦出来一个小小卡，然后你可以在里面去做一些非常小量的低量计算，或者低量业务复杂度的这么一些应用。说白了这玩意儿是不用安装的，不需要因素的，直接就可以打开。所以做这个事儿，也不是他第一天做了，很多人都做这个事儿。但是凭这个chat BT这次就是想再做一遍，但是他这块有什么不同？可以看到这次为什么这个叉PT有可能他能再做出一个，真的做成这个一清应用这件事情？

我觉得核心其实就是chat p掌握了足够大量的很多人的大量的全部的交互内容都在拆GP里面。那这个时候它其实就可以，首先它可以充当google的角色。第二他可以充当分析用户，所有做用户画像，做用户的一个侧写这个角色。而且他还能充当用户的记忆，就是你说了一次啥，他下次再打开的时候他依然记得。那这个情况下，其实就比像百度，像apple store这样的一些苹果的轻应用什么的，会掌握内容会多很多。所以我觉得这事儿，有可能这次他也能做能做一定程度上能做起来，但是做成我觉得很一般。这个值得再观察。

这东西好，然后这个就是它的第一个，就是这个叫什么apps in chat BT这个东西。他其实有几个点是什么呢？比如说像刚才我说的这个。说我想创建一个。这个对，比如说我说一个像我就单纯说帮我做一个party曲子。这个时候它应该会底下会提示你说，有可能你用spotify就可以创建这个东西。

就是它这一块说白了这块是有一个建议APP的这么一个位置的。当然你也可以称为叫广告位，对吧？你可以称他说用掉谁，那完全取决于chat BT或者它背后设定的一个优先级这么一个东西。所以当然我肯定这是一个苹果，不是那个chat BT在在做的一个商业化的一个机会。

然后这个东西它其实再往后延伸会想着什么呢？就是大家很早以前给大家看过这个概念叫LMOS，大圆模型的操作系统这个图，这也是capacity当时说的一个点对吧？就是这张图他认为啥呢？就是以后的大基于大模型驱动的操作系统？

那他其实应该怎么样呢？它能够他应该能像现有操作系统很像，它的结构很像，并不是说它会取代现有的这些CPU、GPU这些东西，它是在上层再搭一层，这一层其实跟现有的电脑很像，那核心不是这个CPU了，而是大模型。然后大模型可以有工具，比如说计算器，python中这个终端这些东西。然后可以来去跟现在的操作系统里面这些应用是一样的，对吧？然后文件系统，那就是现在的这个聊天的内容或者是memory，这些都可以放在这个file system里面，文件系统里面去对标原来的这个操作系统文件系统。然后可以有这个输入输出，这个video的audio的输入输出，这其实大模型现在也都能看到听到了，然后可以联网。这个联网的大模型有自己的对吧？还可以跟别的大模型去互动。

那回过我们来看看现在这一套这套东西是不是就有点像真的他在朝着这个方向来去做了，对吧？那他在尝试现在就在尝试干嘛呢？改造适合于LMOS时代的APP？这样一件事情。

因为你现有的这个应用，比如说在在windows，在mac在手机上的这些APP，其实确实是不适合存在一个大模型的用户。就一个聊天的画面内去完全的无缝的来去使用的。所以他就说好，那我就直接再做一套标准，做一套协议来去跟你看。我在这儿添加这两个playlist，它其实会加到我的spotify的账户里面，然后我在那边就可以直接去这个播放了。但是他现在好像没有，不过因为这个肯定有版权什么的问题，所以他不在这不让在这这儿去播。但是后面肯定他会加这个东西的，只要他把这个商业合作的方式给协商好就行了。对，所以这就是chat BTF in chat BT这样一个想法。

我觉得这个趋势，然后再结合前两天这个open I，前一段已经说了好几个月了对吧？给大家讲了这个open I要做自己的硬件，然后跟谁做呢？大家还记得我记得是上期给大家说的立讯。李迅跟谁来着？戈尔对吧？跟就是他在国内找了两家公司来去做硬件，那一家就是立讯，一家就是歌尔，然后做这个应用。

那大家想想这个东西它的跟这种形态是不是就越来越接近了。就是他希望你在一个一个个界面上去不断的互动。而对于chat p来说，open I来说，有可能他要做的就是一个纯基于语音的这么一个设备来去互动，让它所有的这个内容都停留在它的界面。

那如果这种情况下，我的APP还有自己的界面，对吧？假设我们现在是一个安卓的手机，那来去改造成这个open I的新的硬件。各家应用还有各家不同的APP，那我还得看，那我跟一个手机就没区别了，所以得想想如果open I去搞下一代应用件的时候，他肯定要去动这些应用本身这些主流应用本身的形态，就是他的这个操作界面形态。所以我觉得这也是一个他的中途的一个版本，就是他想去尝试去拿到去这个用户访问这些APP的这些交互界面的控制权这么一件事儿。对，然后刚刚说到这儿，就是为什么说这儿它是有一个所谓的主动或者这个spotify建议，他可以选这个这块我不知道大家还记不记得之前给大家介绍过open，google的这个pixel 10的时候，这个pixel十它出了一个功能叫做叫magic q，我找到。magic. 是不是中文的？

嘿。哪去了？然后搜一下Q这个。对，这是啥东西呢？其实就是这么，就是它可以随时根据你的当前的上下文，就比如说你看这个人是谁呢？就是说嘿你你你有今天晚上地址吗？这个东西出来这个东西你看出来这个东西现在出来这个东西不是人打的，是操作系统层面推出来。他推出来说你可能要回复这个东西，他猜你可能要回复这个东西，等于是个我们一般叫proactive，就是主动性的这个助理来去推荐东西。

你现在唯一要干的事情就是点一下点一下，它就直接真的发出去这句话，所以这就是整个操作系统。它如果知道你足够多的上下文，包括你APP里的，包括你现在看到了啥，在哪儿位置这些东西。那如果这种情况下他知道越多，他越可能给你主动的提供各种类型的上这个建议。比如说像这个，他其实如果能拿到我的日历，那他其实就可以直接不需要我去问他说帮我做一个play list对吧？那他他如果看到我日历里面有一个party，那他应该建议我，我要不要给你拿这个生成一个音乐歌单，他就应该能做这事儿。

所以这件事情大家可以逐渐的发现，所有的主他的主的这个线索都围绕着一个一个词，就是context，就是上下文对吧？就是越来越多的上下文，这些agent这些的这些ChatGPT，这些copilot其实需要的就是你越来越多的上下文。那它自然而然大家希望有一个玩意儿一直挂在你身上，一直能够？耳机，什么手机或者什么新的硬件形态，它一直需要这个东西来去收集你的当前的商，你的所言、所在、所行、所看的这些东西。好，然后这些东西其实说到了这篇文章的最WW点最下面，其实很多地方都没有说到这个东西。我觉得其实这是还是很值得去关注的东西。

这个。Agented commerce protocol这一般减掉ACP东西。A这个东西它其实是放在这个APP in a apps in chat BT这个栏目下的。但是其实并没有很多人去讲这个东西，但我觉得这个是一个特别重要的东西。你看这个有点像啥呢？就是有点像我们在节前讲的这个东西。Google搞了一个叫agent payment protocol，agent支付协议，对吧？因为它叫APP，所以不能叫APP，叫AP two这么个协议，这个所谓的a agented commerce protocol其实就是这个东西。

你看他比如说我想买一个什么样的一个这什么玩意儿，给我的朋友买一个礼物吧啊这个啊然后呢他就会直接蹦出这么一个选择，我们可以直接看一下这个真实是啥样，现在真的可以做，这就是它是可以实现的。这咋还没加载出来，就现在这是我的手机，然后我把它打开，我刚才的这个选择一项。怎么不更新？

稍等，我再看一下。

OK.

最近。

还有安装上还装着。对，那我们直接先看看这儿，待会儿就能，我直接从这儿放回也行。对我当时搜的是什么呢？搜这个东西叫我要一个四十刀以内的大码T恤，对吧？然后他就会出这些产品，这些产品底下给你的结果是什么呢？就是你可以从这些店去买，这个大家比较容易感同身受的。这个没啥问题。但是他在有一些店上，在手机上没法看这个东西，在手在在在在网页上，在手机上这个按钮就会变成一个buy买直接买。

对，就是他已经跟好多家的几家的这种电商网站已经达成了协议。就是它集成了这个所谓的叫ACP的协议，跟这些电商网站集成这些ACP的协议以后，用户可以直接在他的x GPT里面去买。看到这些产品的时候，可以直接一键就可以买到这些对应的商品。对，不知道这个还是看不了，或者大家直接可以看到，如果能直接看到我的屏幕，看这个屏幕能看到吗？也不好看。

对，你看这是什么呢？就是我刚才说的，让他去买那个T恤，然后我就让他去这个所谓的叫什么这个网站叫ESTIETSY这网站去买。他就会蹦出几个选择，你看这个几个选择，每一个选择我直接点开它是什么呢？这么一个详细的描述对吧？然后你往下翻他会干嘛？他会建议你为什么可能会喜欢这个T，反而写一些大模型生成的东西。

但核心是这个，你看这儿有一个by直接买，然后选一个号，选一个号，选一个黑的，然后这就有一个直接by。对，然后他就可以直接去调这个link这家公司的这个支付就可以直接拉起来了。然后他会给我发一个码，然后我就如果过了，他就直接给我扣我的那个信用卡的钱了，所以就是这么个东西。

对这儿说一嘴，link这家支付公司，link这个支付公司是stripe下的公司。Stripe大家知道这个有期给大家讲A正负的时候知道这家公司吧，这个这个很大的一家公司，strike这家公司，这两家公司，这公司的问题就是他还没上市，是ad公司。但如果大家真的是关注到了这个，或者比如他真的要提交上市IPO的话，我觉得这家公司真的是天花板极其的高，就是比paypal高高不少。然后link是strap的一个钱包的一个产品，然后这个方向的机会或者潜在的机会非常大。

然后对他这个所谓的ACP其实就这么东西，它里面都不提技术，对吧？还记得给大家刚才讲的AP two的那个里面讲了一大堆给开发者的讲技术的东西，open I就不是这风格。他就直接告诉你说，他们跟struck合作了一个叫agents ACP这么个协议。你只要接上了协议，他现在就是直接可以用这个SC这个网站去买东西。直接已经接上了之后，正在接shoppy fy正在接什么什么lossy这样的一些公司。他我觉得但凡这个shop fy能接上，我靠又一波又一波猛涨，甚至说可能是对非常大价值的这么一个东西。然后你可以看他整个过程没有讲任何的这篇文章里面几乎没有讲什么底下的这个技术的原理什么的的东西。他就告诉你这个ACP就是用来干agent跟商户之间花钱的这么一个行为的。

而具体底层东西你只要符合这个ACP的这项协议就OK了。你不用太多关注它的这个很细节这些技术，如果你关注的话，你可以看这个整个流程是这样的，其实跟AP two差不多，差别并不大。但是它就是我觉得在这一点上确实open I做的越来越像苹果的这样一个方式了。就是他把这个用户体验或者开发者体验做的封装的还是挺挺高的这么个东西。所以这个ACP我觉得是一个挺值得去大家关注这个方向，就跟每次其实在agent的支付领域里面出一些新东西，其实我都非常推荐大家去跟进一下，这个是非常可见的明确的。这个大的可能未来半年爆发的机会。

对，大家之前那几个其实一波是立讯，其实已经明确了。然后另一波是啥？这个AMD对吧？之前给大家介绍那个AMD那个AI max对吧？AI max那套这个芯片，395那套芯片，这个也可以看出，其实AMD的统一内存这条路已经快走通了。然后在C端已经有一些场景了，有一些机会出来了。然后就在咱国庆期间，open I跟AMD签了一个大单，对吧？跟这个签了一个也没有很大几个billion的这么一个单，几个点这么一个单。

待会儿讲这块东西，大家其实关注一下这些领域的变化趋势，可能也是对于大家看到一些公司有帮助的。然后他这些其实都是关于GPT s in GPT的一些点。第二个其实就是所谓的a agent kit，这个其实就是专门用来去开发这个应用的了。刚才那个APP for chat BT是表面的这个界面，应用端，可以叫应用端。那我后面怎么去构建这个agent呢？其实就是agent kit这套东西要去构建这些，我觉得就一般了。其实可能就三个点。对，这个也是这么三个点。

一个叫agent builder，这个东西不好说，我觉得这个确实之前是open a的一个短板，就是他没有把这个用户非常轻量去构建一个agent这件事给做出一个非常容易上手的产品。而这东西就是这个他这次推了叫agent builder，名字也很直观，你给打开看看，这玩意儿不就是一个简化版的define。对，简化板底翻，然后你把它拖过去，完了确实做的比较漂亮。整个的审美和这个设计的都还是很有讲究的这种东西。但是这里面你看它的内容，他核心推的这么几个，我觉得这个也是挺挺直观的一个点。

就是define，大家看define里面它是怎么定义的，这些节点或者这些工作流，我随便随便开一个随便开一个这个。对，你看，不是找个工作流了。对，你看它它其实重要的节点，它也是一个开始。然后用大模型来去做什么事儿，然后用大模型来去生成什么东西，这儿用大模型来去生成什么东西，最后把哪些东西返回给用户，也是这么个东西。

但是他在这个部分其实都定义的相对来说比较贴模型测。就是每个这样的一个脑子的节点，其实代表都是一个模型调用。而模型调用这件事情或者说比较偏，但是它其实里面已经增加了像memory什么这些东西的很多很多功能了，其实已经很偏agent了。

但是还是我觉得还是没有，其实还没有这个open I这么直接。我这个节点不叫达摩，跟A大模型没有任何关系，都是agent都叫agent。比如这儿放一个agent，这再放一个agent，然后这边直接去取对吧？还可以连到这边来，然后加一个分支键就可以连过来。等于他把每个节点已经又抽了一层叫A键，然后终止，然后寄一些note，然后里面的一些工具其实也比较的很精，什么文件搜索各种的MCP，G real其实就是安全性这些。比如把这个用户的个人信息给包起来，给去掉这个敏感化，然后加一些业务逻辑，比如说e fails，加一些while，用户的允许，这个东西还挺重要的就是你可以加一个节点。这个用一旦到了这儿，用户一定得点一下同意还是拒绝。如果同意去哪儿，拒绝去哪儿，拒绝去哪这样的一些东西。

对，就是这个东西大家看的其实跟define我觉得真的很像，而且功能显然是没有比赛强的。但是好在哪儿呢？或者说open I搞这个东西好在哪儿呢？

它其实是一个全栈东西，它是一个从顶到底的给你的做A正开发的全部东西。包括说你构建这个东西以后，你跑起来，然后我的这个评估，你看这儿它有一个评估，这是团队版才行的这个东西。然后评估，然后后面运行起来的一些资源的消耗监控这些事儿他都帮你做了，就这usage什么都放在里面了。

说白了就是大模型厂商第一方做的define，就是这么一个agent build这种东西。底层的上层的其实都是连接的很顺畅。你想这个eden build a我构建出来的东西，它其实可以直接变成刚才的给那个放在chat BT里面的这个东西的。对他可以直接输出这样的东西输出这样东西。他在他那个发布会的官方的视频里面有一个演示，很快大概几分钟他就给你搓出这么一个东西。而你像define这种平台，它撑死了输出的时候可以输出一个聊天的界面。你可以在里面去聊，然后你想把它嵌到哪儿，你还得自己去写代码，然后你想把它变成API，当然输

出也很方便，你可以直接给到其他的合作伙伴来去接你的API。所以这个大模型第一方做的理发的最大好处，其实就这东西我就可以上下非常无缝的去衔接，这个就是它的agent kid这一套东西。

这agent kid这套东西，我看看能不能找一个他的官方例子。这个官方的视频其实很everyone，this is a from OpenAI，他给你介绍了一个东西，你看他其实就是做了一个什么的，好像是机票问答的这么一个多agent的这么一个东西。就先做了一个分类器，说用来分类什么呢？用来分类用户的问题到底是关于哪类的。

然后他就你看这块写无非就是写写指令，写几个词描述一下，然后输出它输出的有几个分类这几个的分类，不同的分类会跳到不同的A站来去接不同的任务。你看第一个会叫flight，这个航班的agent来去皆第一类的。别的A站会接别的信息的，然后他可以就通过这个if else来去找出去。但是大家注意，这块你看它其实前面分类完了以后会有一个EF而e fies里面啥。So from the flight agent will create.

对这个e false里面其实还是要去自己写，人工写这么一个什么input output pass，然后classes等于哪一个，然后就走这条分支，否则的话走另一条分支，而这东西其实并不是普通的用户。我觉得这个其实真不是普通的一点代码都没接触过的用户就能写出来的东西。所以这个agent builder其实也没有它的门槛也没有那么低，就是没有把它做到一个非常大家都可以用。比如它的八亿用户有1亿用户能用，那显然不是这样一，所以我觉得这个agent builder肯定还要再去演进进化几轮才可能到。

然后第二个是agent builder，a agent这个case做了一个叫connector registry，是这是啥呢？就是大家企业内部署大模型的很大的一个问题是啥？就是我要把企业内的数据接到你的对接到大模型上，或者说我在agent然后能访问到我大量的企业内的数据。

那这事儿怎么做？其实就靠connector，之前其实像cloud什么都已经做了非常多这方面的工作了。而他这次其实就也做了这么个东西，当然肯定要做这个事儿，然后只做了一个叫registered，就是一个注册中心。说白了就是你们这帮做sas或者做外部系统的这些公司，你得自己去主动的去做你的这个sas服务的连接器。把你的这些连接器注册到我的这个registered里面。这样用户在chat p聊的时候就可以直接调用用户本身在你的这个服务上的数据了。

所以这个事儿，其实说白了就是一个准入门槛。就是你这些有数有用户数据的公司，想把你的用户数据对接到ChatGPT里面，你得对吧？在我这儿注册，说白了就是你得你得交点这个注册费，然后你得受我的监管。

然后另外一个有意思的叫什么？叫chat kit。我觉得这是这个是里面最有意思的一个东西。你看这个东西check，它里面的所有的聊的内容，你刚刚给大家看了，这个是一个左右滑的这么一个，我们叫叫vision，叫组件这个东西。这些其实都是这个东西，然后这些东西咋做？就这个vagon，就像听音乐的时候咋做。

他专门做了一个叫wage builder这种东西，比如说，而且这个位置builder里面就有一大堆现成例子，比如说你可以做这样的一些东西，这些都可交互，这些都可以点来点去的这样一些轻应用组件。可以这么说，看这些都是可以交互的。那甚至你还可以这样说，就比如说帮我做一个做一个表不同emerge的选择器。然后他就帮你去做了。

这个东西就是一个它调用大模型去现生成这么一个像这样的一个组件的这样的一个过程。他其实说白了，他在背后写代码，然后你不需要去看那些代码，你只需要看到样子，然后来去确定这个vida是你想要的东西。然后他来去负责去构建这个前端的样式，以及对接后台的一些数据，他还会给你留出一些数据的所谓的槽位所以这个做了一个非常有意思的点，就是他从最底层的这个模型做到了agent builder这样的一个构建。View agent东西。你看这个它就构建出一个region，你还可以点这些都是完全可选的这么个东西。你看这有点问题，它其实并没有写好。

然后构建出这个V这个以后，你还可以换直接下载出的这个vision就是你的这个你要去扔在这个agent builder里面的这么一个前端的形态，前端的一个页面的东西，前端页面一个组件，这样一个东西你可以直接去做了。所以他从open I从最底层的模型做到agent builder，做到这个region，就是前端构建这些轻的应用的这样的一个工具。然后再做到这个chat所有内容方的对接，几乎就把全栈就打通了。大家可以想想就是从底层的infra到模型到agent API对接。然后到agent再到顶上的这些形应用的形态，再到这个chat BT面的内容，就是第三方这些内容植入拆除这个过程全都搞通了。

所以逐渐的可以看出这次这次发布会其实核心他想突出的点就是我已经做了全栈的大模型应用的全生态构建。所以这个还是挺可怕的。而且这个事儿对于国内的大模型公司来说，我觉得至少目前还没有看到一家有这方面的构建的魄力，或者在做这件事儿。我觉得这事儿我觉得还是挺挺值得去赶紧去跟进的这么一个团队。但是谁国内谁适合做，我其实挺愁的。这个事儿我感觉国内好像没谁能干这些事情，可能华为，也就华为能干这事。

好，然后看看这个，这个就是第二个部分叫agent kit。我们说第一个叫apps in chat BT第二个叫a key，它是用来构建agent的。第三个就是writing code写代码，其实这个就是code dex，他把他的code dex第一次就等于算是正式发布了，之前全是内测这样一个状态。

那这是啥玩意儿呢？你可以看一下，我这个就知道了，这个就是code dex。你核心要做的事儿就是我选一个我的代码库，这是专门给开发的人员用的。我选了一个代码库，但是大家不用担心，这边跟代码没啥关系。然后把我想做的事情直接我选择它来帮我，比如说我选这个帮我做一下call review。找找有没有严重的bug，尤其是性能相关的OK然后他其实就针对我的这个代码库去搜了，这是在get up上的代码。

然后你看他就会新建一个任务，这没有的任务新建一个任务。这个任务其实就跟我当我现在说白了，我现在说完这个命令，我就可以立刻关机了，我就可以把电脑关了。然后他就自己在云端干活了，根本不需要我这台电脑。你看他就自己去起了一个环境，然后自己去干这件事。然后搞定了以后，你的那个手机的APP就可以收到一个提醒，就完了。你想你想继续去让他进一步的干别的事儿，你就得回来继续去跟他说，就这么个东西。所以这就是他的codex。然后据说codex最近的又猛增，比clo code什么这些的用户量增加多的多了很多。

为啥呢？就是大家看这个code dex这个产品最大的点是什么？如果大家有程序员朋友的话，知道程序员平时咋干活的，就是打一个curse对吧？在这写代码，写完代码在这个命令行里执行运行OK了。但是如果你看刚才我这个任务咋做的，我这个完全是靠自然语言说的，完全不需要说任何的代码，直接我就给他发号施令就行了。说就跟那个非技术的领导是一样的，直接给你就直接发发命令，就OK了。然后你可以甚至可以说我马上双十一了，然后一定要测好这个压力抗压程度，这一些东西都OK了。

所以这个code dex使用的过程就是越来越偏非程序，或者说叫越来越偏非直接的coding的场景，我觉得这个是一个特别明确的趋势，前两天cloud也的老板很反华，那哥们说也说他们内部几乎已经没有人去写代码了。那人类真的直接写代码了，大部分的人都是在这个叫什么做review的过程中都在干这一件事情。所以这就是他的第一第三个这个code red code部分这个点。然后第四个其实就是他发布了几个新的API，什么sora 2，然后这个GPT5 pro，其实都发出来了。我觉得这事其实刚才一直想给大家看到那个sora的那个样子能打开了。反正。怎么怎么刚开始可以又不行。算了，这个sora，我们直接看sora的官网吧，sora。

对，说白了就是solo二这次的这个效果真的是非常好。而且他还尝试用sol做一个再上刚开始尝试用它去做个社交产品，或者说白了打起了抖音，那你看这效果，我的网怎么这么长，对这些效果你可以看这种东西，你看这个他是直接弄了一个滑冰的，举这个猫，确实整个过程的物理符合度是已经很高的。你看还那个猫掉下来能看到这样的一个滑一下的，然后这个人什么在在滑板上往下摔一下，整个衰的过程中，这个水是会就是他虽然在滑板上跳，但是依然能够模型生成的时候知道说他摔下来的时候水花会被溅起来，说白了就是啥呢？他就越来越

越懂所谓的物理了。你看俩岔开了，两脚叉开了以后会掉下来。这些就是他对物理世界的了解是越来越真实的。

所以sol 2他号称自己就是这个叫什么物理模型，世界模型的下一集下一波这样一个东西。当然这个布世界模型跟之前给大家介绍什么VJ pa这些还是不太一样的。当然42他也没有公布它的具体内部的这些架构，到底咋做这些东西，那依然是一个很close的一个状态。

然后他去年搞GPT421的时候，他就号称到了这个视频生成的GPT1时刻。然后他现在就号称sorry 2，这个玩意儿就是DBD3点50克，我觉得确实也挺像的。就是你想大家想想这个B3.5就是发布ChatGPT的时候，对吧？那发布ChatGPT的时候就基本上到了普通用户可用的一个状态了。如果说说20的话，那我觉得也不足为奇。就是你像这种视频，我都肯定普通人是可以接受的这样一个状态。

你看他跳了一下，把那个地上的，把那个鞍马上的一个粉给震起来了，已经是很有这个世界母亲的意思了。这个就很奇怪，一个马站在另一个马上，但是他的点是什么呢？就是上面这个码其实是不用走的，对吧？所以这个还是他认为这样的生成的结果是正确的，是符合物理事实的这样一个东西。

对，所以这个底下这些例子就不说了，然后他生成了一大堆这种东西，新生成了一大堆这种第一人称的这种视频，就跟tiktok的竖视频就很像这种风格。然后这种视频就做成了他所谓的42这个APP。然后这个APPP的打开，直观的会想到说这样是不是对标这个tiktok的。他还专门写了篇文章说，说我们sora这个APP的feed就是那个那个流就视频流。

这个视频流的哲学，说白了背后想说的就是我们这个sora跟这个tiktok不一样，我们不是为了这个唇让大家娱乐的，所以他我觉得其实硬凹了很多，硬凹了很多。我不知道大家有没有这个，如果看的话可以去看一下他引来了一些什么呢？他说我们这个他说他们这个骚扰是为了优化创造力的，而非所谓的叫passive squally，就是被动的滚动。所以他就说这个骚扰我们是让大家更更有创造力，更能够创造新东西，而不是为了让他脑子变笨这种东西。然后所谓的优先考虑连接，就是让大家能够连接起来，能大家能社交起来。所以这里面提了一个他们现在这个产品的功能，我觉得很有意思叫camel camel这种东西。但是我的屏幕显示共享不出来，看能不能勾勒出来。

好了，对，这就是刚才那个我直接在这个APP里面可以直接这样买，直接点这个by就可以买这个东西。然后这个蹦出来的东西，就是它的一个算清用吧，那个东西然后看sorry。

然后注意这儿左下角这个小小标咦。怎么网络的又断了？

连着有线都这么慢。

现在好一点。对，就是这个网速我就不知道是怎么搞的，他们极其的慢，稍等我把这个换成。

我看看，那我算这样。对，可以看这个。这个是啥呢？就是我基于一个我只要上传一个我自己的一小段视频，这玩意叫凯缪。这个added camel这个东西，我只要上传自己的小段视频，那段小的视频贼短。就是你自己念什么三个数，然后上下左右晃一下脑袋就OK了，然后就形成你的一个所谓的cameel。

这cameel就是这个东西，我只是选另一个人的那个camel，另一个人camel我直接说这个人抱着一只猫从太平洋里直接蹦到了火星上这么一个过程，怎么加载不出来呢？然后他就会真的变成那个人的脸所生成的这个视

频，对吧？这个玩意儿就是每一个人的camel。所以说白了你可以把任何一个人，比如我做了这么一个人们在梅花桩上跳来跳去这么一个视频，咋弄都弄不出来。

他这个真不是我这儿的网的问题是巨慢，就是我的网它再慢也不真正慢的，肯定是是是这个sora它本身的官网的这样的一些网速的问题。但你可以看到这儿，这个小窗口里，从这个小窗口里可能大家一窥一二，这个就是。就是我生成了几个视频的这么一个点，上面抱着一只猫从太平洋直接蹦到了火星，然后这个人们在梅花桩上闪电般的跳来跳去。

这些东西里面你可以换脸，可以换成什么脸呢？再也不是以前那种直接你扔一张图，然后直接给你生成这个图背后这个脸，这个明星的这个脸。而是每一个人都可以把自己的脸先设置好。然后你要觉得有意思，你就可以换成你要觉得你生成这个视频有意思，你想去把别的把你的朋友也拉进来。你可以直接把对方的这个生成那个camel，然后直接可以引入到你这个视频里面。所以他的意思就是可以通过这样一个生成视频的方式来去做社交。

做社交这个事儿其实在好号称这个chat BT sorry发布之前，他们内部我上线了一个功能叫做什么upload yourself。其实就这个东西把你自己上传上去，然后大家互相的用别人的对方的这个脸来去生成这个视频。当然这个过程你可以去设置很多你的隐私方面的一些设置，这个可以让别人谁能够用了谁不能的脸这样一些事对，这就是骚扰骚扰我觉得挺有意思一点，就是他在尝试做一些已经在尝试做一些新的社交的行为了。这些行为我觉得大概率还是一个内部的玩票的一个行为。但是这个OKI如果想搞这件事，他他8亿用户怎么着都能够搞出一些想。对，所以这个就是前面这个骚扰这个部分。行。所以整个的过程，像从APP的APP chat BT到builder到building这个agent kid到code dex到最后几个API都已经给大家说完了。

这次的这个发布会的这样一些内容，大家会后面有一个我觉得这个比较意思的观点，这个是一一个是当然这些事儿不是将来不是一次了。他最早做plugin应该23点。然后后来做chat BDS GPT s也是二三年，还是应该是2424年。对，然后到这次做APP in chat BT，其实这个叫什么做操作系统这个事儿的心一直没死，而且他必须得做这事儿。因为他的这个角色就是我拍的这个角色，他这个估值他已经把它架在这儿。

他必须做非模型以外的整个垄断市场的一些行为，才能够撑得起来他的这个估值。你看这个人的几个想法，包括这个agent kid，可能是一个start up的killer，就是创业公司的杀手。这肯定是的，这个毋庸置疑。就是每次大模型厂商的这个叫什么，大模型厂商的进步其实都拓展边界，都是创业杀手，创业公司的杀手。

然后这个叫什么turmoil gy就是名词的战争在继续。这agent是啥？大家看到刚才那个agent builder有没有？那个agent builder很大的一个问题就是agent的定义。大家已经一段时间已经提的越来越多叫agents。Agents意思就是我不去做那些应用的工作流，我要去做一个让agent自己来去决定怎么干活的这样一个系统，这个玩意才叫真agent。而agent builder搞出来什么又搞出工作流。所以他就说这个名词的战争还会继续，大家肯定是有点开倒车的意思，就是open I搞这事儿有点开倒车的意思。对就没啥了，这过年就找事了。最近国内的几家大模型厂商出的深度研究，远程电脑，远程电脑输入需要直接出代码和实现，这个评估一下，评估一下影响，就是国内这些deep research和这些GY agent对吧？这个能评估一下影响吗？

这个影响其实之前几节课有讲过，然后deep research这个方向几乎我觉得现在是啊我个人感觉比就国内的这一类的deep research这一类的agent是比美国牛逼的。你看像这个minus James Spark这类的国内出海的公司，包括国内这些存在国内的，像什么阿里出的几个开源的，然后字节的这几个cos，字节也有几个什么什么flow，dear flow什么这几个deep research都已经是非常强的deep research这个框架的。但是这类的框架我觉得很大一个点就是内容不好，就是他能搜到的内容有问题，大家这一点也是很早一次课给大家分享过。就是你像perplexity这些产品，这些产品它有多强的A的能力吗？大家如果用一用，就知道其实很一般，或者它其实对标的是搜索引擎，至少policy它是要对标搜索引擎的。所以它在所谓的这种research方面其实做的很一般。那谁做的这个好呢？我觉得还真没啥。

你像这个ChatGPT的这个agent模式，这个agent模式也很一般。它deep research模式还好一点，但是d research模式最近我也发现它这个生成内容越来越少，或者说越来越不全面片面。所以我觉得这个agent这件事，这条路还得靠国内的一些团队来去搞。比如像这个je Spark对吧？这个James mark就非常优秀，那个材料我不知道有没有有人用吗？James Spark百度的几个人出来找了一个公司，这可以看到啥呢？他就走另一条路，就是我不去选这些什么纯通用的这个agent对吧？当然它有一个叫super agent，就是你可以你不知道到底选啥干啥事儿的时候，你就直接选super来去用。然后如果你知道，你就选具体那些，比如说做PPT的，做表格的，甚至做设计的。

然后后面还有什么做视频的、做图的、deep research的，还我觉得他专门做了一个agent应用，叫fact check，就是事实检查，对吧？大家经常会有一些看到一篇文章？什么保健品乱七八糟这些东西，震惊了，到底是不是真的？他专门做了一个AR来干这些事情，所以这就是从通用到专用这类的A证都有非常多的都做挺不错的这样一些应用。

包括这Spark，包括对这个我觉得反正现在deep is deep sick这些已经大家不要用。如果真的要干活用的话，就不要用deep seek这种官方的应用了，就直接用这个像什么minus包什么，其实都挺不错的。然后包括质朴的这个质谱，这个叫Z点AI的，这个也是做了一些很多东西。比如说最近刚上的4.6，什么都能做很好的从deep research到应用构建这些工作。Kimi那个那个A站是吧？

那kimi那个agent我们下次讲的话，我觉得那个确实做的很不错。这个有有的有的讨论，有的聊。对，然后可能看到3D打印机HRS这个对我我有一个3D的叫什么拓竹的叉1 carbon开源图纸库，其实就是make up，make up什么呢？Make her words make a word。它官方的，然后我不知道你是不是看了之前有一个拓竹去告哪家那有一个哥们儿去当了很多他这个官方的这些模型，然后告诉他他是不是你看了那个事情，就这个3D模型领域里面license我觉得还是有有有很多要去考虑的地方。它不像什么图片文字什么这些版权这么清晰些。所以如果真的要商业化的话，还是要仔细的考量一下。当然如果你就做点小事儿，那就没关系了，这个不用太操心者。

对，OK然后看看这个是什么。对，然后这个有一个网站很不错，这个叫small SMLMSMOL，small ai这家公司好像是也是YHUZ投的，还是YZ投的。好的一个家公司他是做啥？其实就是做AI领域里面的新闻解读，可以叫新闻解读这种东西。然后它好到什么程度呢？我觉得就这一点可以看这个东西，这个是某一天的某一个事件的这样一个总结。你看这是他的总结内容，它会有大量的tag，这些肯定都是它的人工加AI识别出来的整理出来的东西。然后底下的内容非常全，而且很干非常干。

你看他这个从def day的这些很详细的一条内容，就是一title这个链接title链接几乎刚才我们讲的东西都在里面，而且他选的都还是挺官方的内容，或者说选择高质量内容。他从前面的这些这几个核心点，到后面又开放了几个模型，对吧？这些列的都很细，基本上我很少见到国内的媒体能够把这些东西说的很全的，就是这种很技术上的说很全的这样一个内容，包括这个solo 2的prom guide的什么的，都说的非常全。然后他通过除了这个官方的内容以外，还有这个社交推特上的这样的一些总结。你看他会根据一些比较重要的一些相关的人的推总结出来几个点，然后还有一些其他的一个新闻的这个要点总结的都非常好。我觉得这个是small AI，我觉得这个产品做的非常好的一个地方。

给大家看一下，这个我觉得还是挺值得讲的。一个是这个就是之前给大家介绍deep sick 3.1的时候介绍了一个人，说到了一个。推这个人叫computer king，他做了一个分析，做了一个为什么3.1要搞一个。我不知道大家还记不记得，叫UUE0UE8M0。这样的—一个数据的格式，就看看是不是这U18对UE8M0这个数据格式提到这个人，这个人分析的非常的深度，对吧？

最近他又开始说一些别的，就是整个AI领域的这个赛道，中美现在走上一个非常不同的两个叙事，一个是啥呢？就是美国的叙事就是几乎全all in AI，然后啥你看这个数，我当时其实挺震惊的，AI领域的投资占了国内总投资额的接近一半，就是你说一家一个国家接近一半的钱全投在大模型，说白了就是大模型的AI就没有别的太多的AI的这个出来，或者像plantier plantier这种也算但是更多的其实绝大部分都在投info部分。这个是这样，而整个AI相关的股贡献了标普500它近一半以上的增幅。所以就是前段时间说如果但凡美国的AI增长放缓了一点，都是股灾级别的影响，非常可怕。然后这个而中国其实在战战战略重心转向黄金，这个我我我不专业，所以这个小断肯定是讲过很多，包括中国大量的做储备。我不知道这个大家反而会有很多这种消息，大量做一些战略物资储备，这些东西包括黄金，包括一些油或者什么石油，都在做储备，然后中美其实在这方面做了一个非常大的一个变化，然后就说到什么呢？

就说到deep seek发了一个叫3.2 experience，这个应该是实验的意思，这个3.2的版本，然后3.2版本他提到了这个东西叫tee long，tell long这么个东西。Til long是个啥呢？Til long可以理解成这个东西，他这句话就是这是deep sick官方这么说的，til long跟CUDA算子开源的这个新模型研究中，需要设计实现很多新的GPU算子，说白了就是大家这帮做大模型的公司，他会设计很多新的计算模型的公式。这个公式怎么跑到GPU上？它要去做一些算子，这些算子就是用来运行这些大模型的这些计算，怎么跑到GPU上的这样一些核心的算法。比如说比如说比如说我说一个4乘4的矩阵乘4乘4的矩阵，这个东西如果我想去把它在硬件上做加速的话，我就要做一个GPU的算子来去实现这个4乘4矩阵的乘法这么一件事情。好，这件事情其实就是所谓的苦大蒜子的开发了，有大量的工作都在做这个算子开发。

而til long是干啥呢？Til long是一个更高层的一个语言，可以看这张图就知道了。就是对于一个开发者或一个专家来说，他不用去直接写库大级别的算法代码，扩大的代码都是C加加或者C写的这些代码，而直接用这个tile long级别的代码，这个tile long级别代码都是python，是吧？不是，是他的。对，它是叫类payson的一个语法。就是它定义了一种自己的DSL，这定义一种自己的语言。这种语言是累拍摄的，说白了就是比较直观，就是写的比较简单。所以你看他说用80行代码就达到了官方500行代码的性能，所以这个tile line其实说白了就是包在扩大上面一层的一个酷。

那为什么这件事儿特别重要呢？就到这儿了。你看他要浪在前段时间宣布了全部的已经在支持升腾的芯片，就是action c就是这个升腾的这个库达这套语言，然后NPU的这个分支也都支持了，所以他直接官宣了升腾零倍支持deep sig v3.2的这个版本。为啥呢？就是因为DCV3.2的这个版本是用了til lag来去调用底层扩大而升腾或者说叫til long，它默认支持了升腾你的底层的这些语言，A选C这套东西。所以大家可以看出什么呢？就是为什么要这么做？

肯定最直观的想法就是我们先把上面包一层，然后再踢底下的东西，对吧？这就是一个最最常见的程序员去重构一套系统做法。就是我不直接改你这头发是山，或者不直接碰这个头，非常可怕的一套东西。我先在上面盖一层，这一层我把所有的我的应用调用的都对接到我这一层里。这一层专门有一波人去适配你现在这坨屎山。好，那同时我还能干嘛？我适配一套非库达体系的底层，我也可以很方便了。因为这样的话我以后但凡底下比如适配生成这些东西搞得比较好了，我上层一行代码都不需要改，我立刻就可以直接迁过去。

所以其实这是tile long的这么一个意义。这套东西也不是中国第一次搞，这个程工程上其实大家的思路都差不多，open l在好几年前，应该21年，搞了一个叫吹烫的东西，基本上也是这样搞的，也是一个开源的，然后上面架了一层。最后接科大这些这些厂商，GPU厂商也都follow了他的这项协议，所以也都变成了这个标准的之一，标准的部分。而tilt基本上可以定义成是对标是中国的吹捧，就是欧派的吹干的事情。

所以可以看到其实这个问题是啥呢？就是大模型厂商他在越来越多的做全栈的优化。而只有而而且只有从大模型厂商出发，才更容易去做所谓的这种国产的软硬件协同这件事情。所以大家说寒武纪已经被炒成这个样子了。但是我觉得其实还是更多的创新是应该从这个大模型厂商。因为大模型厂商出发，他的点是他的需求是最直接的，他最懂需求。而你让寒武纪去做对着大模型做优化，那这个其实是一个我觉得有点背道而驰的这样的

一个优化方式，不是做不了，而是效率肯定会低很多，所以这件事情其实也代表了我觉得非常好的一个趋势。就是所谓的他说的国内国产的软硬件协同设计，终于开始一个接近全站的一个过程开始做了。

那这个事儿其实涉及到啥呢？就是treat，不是那个til long这个玩意儿。Til long这个玩意儿是谁做的？是北大的一拨人做的，telling这是北大的一波人做的。而这波东西其实就涉及到我刚才底下这个组织的和人的部分，就是tie AI til long的这些contributor。你看现在有七十多个contributor，说白了contributor就是给这个代码贡献给这个库贡献代码的人。这波人我觉得是不亚于这个deep sick的这波做info优化的人的价值了。所以一定得看好这帮人，对吧？

王磊好像就是北大的那个人，然后这个也是成语PKU也是北大的，核心的几个人都是北大的对，所以这个tilling真的是大家看到了一些这个产业跟学术结合的全栈软硬件优化这样的一个机会。的进展。最后下一个说说这个文章，这个文章是很有意思的一个，这是彭伯格的彭博社的对吧？大家知道这个彭博社有时候臭名昭著，就是这个乱说，我咋打不开呢？一定要交钱吗？对。我靠。不能看了吗？

彭博社有一个这篇介绍，介绍的非常好。他就是关于说大家前一段时间说了好多次open I跟NVIDIA跟oracle做的这个叫什么，前后吃的这样的一个合作，对吧？这张它里面有张图讲的非常的好，我咋打不开？好像可以这样。来看这张图。对，这张图我现在看这张图，我现是我现在看到的讲这件事情的最清楚的一张图，而且最全面的一张图。你看这个最大的这个圆中间最大的圆是谁？是NVA4.5个child，就是4万5000亿4万5000亿市值。四对，4万5000亿市值，最大的一个比比微软都高。微软现在只有3.9个群好，open I在这儿，现在500个卑劣也挺大的了也非常大的。

这个线绿色的线，绿色线代表的是风险投资，你看就是他投了这些公司，比如Harry大家知道吧？就说了好多次了，法律的AI，然后这个amb ambience什么，这些都是医疗类的，他投他反向换的什么呢？换这些公司来用open I的API，就是软件或者硬件提供服务，对吧？等于是我投了，反正你肯定要用我的模型，你肯定要把钱还回来，他跟前段时间跟这个oracle的这个生意就是这样，就是NVD投open I100个billion，然后open I承诺oracle 300个billion，然后oracle再买很多billion的NVA的卡，对吧？是这样的一个点，大家总觉得好像这玩意儿就是一个前后吞，但事实上整个市场上并不只是他们三家在这么玩，这么多家公司都在搞。你想这个core wave，core wave是一个前端非常IPO的一家很有名的一个公司。之前也就是基本就是挖矿，我觉得就是比特币挖矿的这些东西现在不知道多少，大家可以看QV或越来越高对吧，还跌了一点对吧。

但是随着这一波大模型厂商跟云服务厂商互相的勾结，可以叫勾结，也已经涨了很多了，然后里面有几个小的，可能大家会如果大家做美股的话，可以关注一下。像刚才说core wave，像navius，navius也是一个从挖矿出身的公司。然后这家公司，原来是好像是，对，好像是俄罗斯的一家公司衍生出来的。后来直接就变成了一家专门的云服务厂商。这个云服务厂商，你看跟微软，跟video都其实都很关系。它有一条蓝色的就是investment，就是MVR厂商，NVD都很喜欢投这种云服务厂商。

因为我投了你，你回过头来买我的卡。Open I也是我投了你，你回来买我的API对吧。然后这个稍微麻烦一点，就是open I跟GPU厂商对吧？

所以这个就是因为他不能OKI，他直接买了卡，他不像XAI似的。XAI自己买卡，自己建这个集群。OPI这个体量比较大，所以他买完卡他还得找人建集群。所以他要不找微软建，要不找core web，要不找nabbs，然后甚至像这个，因为in skill这个是一家做挖矿的公司，大家可以关注一下，这家公司好像没事，就是是没有。对，没有，对，但是这些公司巨年轻行业一年多才对。所以这个都是这么搞，我能给你我能投给你我能确保投完了以后，你会钱返大量的返回给我，所以这件事情就跟AMD这个就很像，就是open I开始跟NVIDIA跟oracle达成一个三角，然后AMD也是这么搞，对吧？

就是open I直接去。这个时候他说要自己去部署这个AMG的GPU了，那这个时候其实并不是，我不知道他不是找第三方来去做，但是我至少看到公开资料是他是OPI需要自己去布的。你看现在大家去号称的买GPU都已经怎么说了呢？已经不是说什么买什么十万张卡、20万张卡是拿叫这个what g瓦特，就是我要布的是这么多G瓦电量的GPU，已经不在乎到底多少块这样一个东西。当然它主要据说主要部署这个6G瓦的GPU，不是为了推理。所以这也跟咱们之前给大家说的这个AI max这波的机会还是挺像的。其实就是他也不会去指望AMD能够直接去取代NVDA作为他的训练的基础设施的取代方。

但是推理是真的可以搞的，然后AMD反过来给了啥呢？AMD给了open一个option，可以叫机会，一个叫期权然后给他这个机会可以干嘛呢？可以买好像是10%的AMD的股票，而且以一个极低的价格买他的股票。当然它有条件，就是你必须open I买了我多少AMD的卡以后，然后达成什么样一个里程碑以后，你就可以去买我的AMD的股票。所以这玩意儿就变成了这样的一个循环。我有需求，然后我买你，我投你，你再买，还有我这样一个过程。

大家都是这样，你看一圈一圈都是这样，所以布隆伯格这篇文章就讲了很多说关于说这个东西真的是其实潜在的风险真的非常大。你但凡这个环节里面有一个掉链子，当然这个掉链子其实主要指的是像这种大模型厂商或者应用厂商。因为最终的整支撑整个估值都是靠它的应用方，说白了就是消纳。如果但凡有一个消纳方出问题了，或者放缓了，整个循环都会有问题。反而是像这些云厂商。你少一个扣少一个n skill，还有另外一个，然后少一个什么figure AI，其实还有别的，其实这还好。所以这张图我觉得挺有意思。

就是我第一次看到这个写的相对来说比较全的这么一个东西，这是bomberg的一篇文章来去讲这个东西。我看有啥问题没有我觉得百度地图做的挺好的。现在旅游都是茶景点，而且分单会给出平台价格。

对，你像高德什么的，其实不都在做这件事儿。但是我觉得有一点就是你还是刚才我那个说法，就是我们在一个百度地图里面去，百度地图。在百度地图里去搜内容的时候，或者说你的筛选条件，你只能按他的筛选条件，他给你做筛选条件，你就只能用这些筛选条件来去筛，对吧？

比如说我说这个奥森是我没法说，比如说我想奥森附近的捏脚垫，不对，叫叫什么洗浴中心。对你说这种东西是他也能搞上。那这应该是找到那个洗浴中心的关键词搞出来东西，而且是奥森附近的。那你看我往下挪一下就不一样了，它其实还是按照关键词来去搞，对吧？还是按照关键词。比如说我到这儿来去搜，那他还能搜到，那就是两关键词一拼。对，但是我如果说比如说距离距离故距离科技馆的路程五分钟洗浴中心，看他那么搜的，那就没辙了对吧？这个东西其实是纯靠关键词来去搜索东西了，他都给我变了，他都把我的query给改了。

所以这个是我觉得像刚才那一类的产品里面最大一个点，说白了就是什么呢？我们有可能可以有一个全定义的、全自定义的对现有的数据库的筛选分析搜索的这么一个功能。那这件事情谁能做？肯定不是。

比如说就是百度地图这个团队肯定是搞不了，当然他可以用大叠加大模型来去做但是相比百度地图和比如说deep sik这样的团队，我觉得肯定是deep sick用百度地图的API做成这件事情的概率，比百度地图用deep seek的这个模型来去做成概率会高，而且用户的体验会更好。因为百度地图它是有自己的传统的这种交互界面的局限，他必须得搞成这个样子。对他这个各家其实都做的挺好的，就是纯从导航来说，现在各家都做挺不错的，很不错的。好，然后大家有啥问题随时发。我觉得这就是今天叉号有事儿，一直没来，然后我就哔哩吧啦一直在给大家讲，这个可能也有点无聊，所以最后讲的这个模型，就是大家知道现在大模型用来写代码对吧，就是code模型。但是还有一种叫世界模型，word model对吧？这俩东西拼起来了，就是meta出了一个叫code word model代码世界模型CMW这么一个模型还挺大，32B这么一个模型。

这玩意儿是啥呢？那我们看啊就是我们说传统意义上一个agent怎么帮你写代码，咋写呢？就这样一个过程。这个如果是一个agent的推理方式来推理，就是我让他写个代码这个问题给他，然后他思考开始写对吧？写扔给一个世界，比如说写推断代码，然后不知道能不能跑跑的好不好？扔给一个世界，这个世界对于code来说就是一个运行环境，对吧？

这个可以跑跑代码的一个服务器，然后这个服务器跑出来结果对或者错，就会有产生一个环境的反馈。他会根据这个反馈再去思考，如果这个反馈失败的话，他再思考，然后再产生行动，然后再扔给他的世界去执行，再收集反馈这样一个过程，这就是gently reasoning。所以整个过程你看它的大模型部分参与到哪呢？Action这个到底调啥？Action肯定是大模型参与的对吧？然后根据这个thinking，根据反馈来去思考，前面思考也都是打磨起来去干的对吧？所以是干这个事儿。

所以这时候的模型就需要核心的能力是啥？是他要能根据你的问题去调用这个工具，他只要输出那个工具就行了。然后他能根据反馈的结果来去思考下一次再调工具这样一个过程。

所以它的模型本身更多的不是它是没有那种我执行完以后，这个世界到底会发生什么改变？这个环境到底会发生什么改变？这个方面的能力是知识水平比较弱的，就跟大模型大元模型一样，对吧？就是你跟他说个什么话，然后他给我生成什么文字，生成篇文章，那很OK。但是你如果让他去想象这篇文章给比如说给咱这个直播间的同学读了以后啥反应，或者说发表到了，比如发表到人民日报以后会有什么反应。这件事其实是他这方面知识很弱。他就想另一个点就是我们可不可能搞出一个模型呢？把刚才的这个执行action到拿到世界环境的反馈这些事儿给训进去了，它就变成这样的一种模型了。就是说我的问题过来thinking，thinking完了以后是干嘛呢？

弄到一个世界模型里，这个世界模型不需要去直接调动调用外部的环境，他可以想象我如果这么干了以后，整个环境会给我什么反馈，或者说这个执行结说白了就是执行结果会是咋样。再有想象，然后再想这个想象这样执行完以后会有什么反馈，再想象下一步操作咋呀。直到最后我就可以想象着，如果OK了，整个我靠想的靠我这个脑子里模仿运行的整个过程，如果能work的话，好，那我就直接认为这些行为是对的，是生成最后的这个行为的结果。所以说白了就是这个word model跟前的区别就是在他输出之前，他已经在大脑里面想象了很多遍。整个这个行为操作以后的结果，就跟大家之前想那叫什么来着？那个那个漫威的奇异博士对吧？什么我已经测算了什么什么几亿次的过程，然后每次结果里面只有什么什么百分之万分之多少，这个只有一次能成功，这意思就是它有一个巨强的word对世界逻辑有知识的这么一个模型去自己推演，最后产生一个结果。

好，那这个这个能在代码里为什么特别有用呢？就到这儿了。就假设你现在有一个代码说这个任务就是数R就strawberry里面有几个R这个经典的老问题。好，这个问题如果我想象到，我设计先拿这个东西，这个任务生成一段代码，就是这个count这个函数，这个很容易做出来，很简单。好，那这个模型，刚才这个可以自我博弈的这个模型它干嘛呢？他就会去根据这个任务以及这个代码来去想象出每一行执行完以后可能的结果是啥。

你看它如果第一行，他是跑到第一行，第一行是定义这个函数，然后这个函数的S就是它的输入strawberry，然后这个要检查哪个字符呢？就是第二个T这个参数叫R好，那他会想象，我这时候这个参数是这样的，然后执行到第二行了，这个N等于零了。就到这儿这个函数第一行里面是第一行N等于零了，他就会想象没变化。好的，然后第二行for c in s就是循环这个字符串里的每一个字。然后他这边就会想说，这个时候S和T可能都是一个什么什么状态不知道，或者一个比较复杂状态。但是这个时候的N已经等于零了，为什么等于零呢？是因为它之前一步它结果action的结果就是N等于0，然后他就会继续去分析思考吧？

它每一个C是啥？每一个C下一个C是吗？第一个C是S第二个C是T第三个C是R，就跟咱学写学编程的这个初始开始这个初初初这个叫什么？

刚开始学编程的时候一样，老师肯定说你想我现在执行到了这儿了，我现在执行到这儿，我现在执行到这个循环了。第一次循环的时候每个值是多少？第二次循环的时候每一个值是多少，对吧？就跟人想象这个编程是一样的。然后他通过这样一个模型以后，就可以大量的生成这样的一个数据，以及他最后思考的时候也可以这样去思考这个东西。所以他就说怎么训练呢？通过预训练，通过SFT，通过强化学习，加起来训练出这么一个叫代码世界模型。

这个东西挺有意思的。这个怎么保证想的是符合实际的？这就是刚才说的，就是他的后面开始预训练肯定是学了很多，不只是代码相关的，什么科学乱七八糟什么知识都有啊，就这块还会有预训练，训练多少八个T的token，然后搞出了一个上下文，好像是8K这种东西。然后又是中期又训练一些后面这块强化学习，什么监督微调这个部分，我不太记得他是哪一部分。加了很多很多的数据，那些数据是哪来的？就是刚才这种就是我执行一行这个东西啥，就我每执行到一行以后，各个变量的变化，这个值是不需要人去标的。大家如果学会写代码就肯定知道说这玩意儿无非就是我断点打在这儿，执行一下这一行到下一行。

那个那个叫什么编程的工具，是可以准确的告诉你我现在执行到这一行以后，每个参数到底是等于多少的，非常精准的，我绝对不会错的那但凡有几行代码疯狂的执行过程，一段时间以后就可以生成大量的我当前这个action执行到了这儿以后，我当前所有的变量应该是多少，又执行到这儿以后，我所有的变量都是多少，这样的真实的准确数据。所以这就是他用了大量的数据，怎么产生这个大量数据？我猜应该是这儿就是100个token可能都是从这儿来的，或者后面也是。对，这厢学习应该是从强化学习部分来的，就是170多个B的token，一百多个B的token，这肯定不是什么代码库什么生成的东西，肯定是大量的合成数据，那合成数据其实就是这个部分来的。所以就是如果你给一个模型大量的真实的跑这段代码，每一行跑出结果的时候的。那个数据这个数据就可以拿来去训出一个几乎是可以保证这个比较符合事实这样的一个模型，所以这就是所谓的CWM对吧？

大家又听说了一种新的世界模型。好，然后底下那几个快速说一下。第一个是那个chat BT boss，这个boss我也是刚给我开通的，就是每天会有一个号称会根据你之前chat ChatGPT里面聊的啥，搜的啥以后，主动的给你每天晚上做总结的这么一个应用。就是你不用主动的问，他会根据你的干了啥这个事儿来去主动的给你题你可能需要哪些东西。比如说他之前发现你搜了很多什么关于什么台湾的某一个活动，这样的信息他就会帮你去分析，帮你去建议你可能需要知道更多这方面的信息。然后对，就follow up这些东西。其实他说白了就是会根据你平时白天干了啥事儿，主动的给你总结你第二天可能需要哪些信息，这样一个东西。

所以这个又回到最开始跟大家说像包括OPI的这个设备，这一点就是上下文是极其重要的那chat笔记本，大家现在很多年轻人、小孩，美国的很多小孩整天都有chat，里面已经不用google了。那大家可以想象这个上下文已经是啊对于这个用户来说是多么重要的这么一个信息了。那欧派I绝对不会放掉非常宝贵的一个上下文，对吧？那欧派那个硬件肯定也是获得上下文用的。所以这个事儿，其实大家也可以去当做一个标准。就是你看哪家大模型应用公司最可能跑出来。那就是看到底哪家公司的产品最能广泛的收集用户的上下文。还是context这个非常重要的这样的一个标准，我觉得这绝对是不会错的。

第二个就是gym I google的这个gm模型能干嘛？这玩意儿可以看地质图，地质的这个遥感图，我不知道大家有没有做这个方向的，我也不熟，但是他的大概意思就是什么呢？就是你的D如果用一个，因为大家知道这个大模型，多模态大模型，其实就是看RGB图的，就是看游戏就什么这种红绿蓝颜色的这种图片的那你但凡来一点什么红外，乱七八糟这些数据，其实大模型是就普通的多模态模型是看不懂的那他做了一件什么事儿呢？把这些多光谱的这些频段的这些图的样子，就是内容转换成RGB图片，把它转换成这个普通多模态模型能看懂的图，在辅驾辅助以专业知识，比如说他把一些什么什么偶像什么来着，一些什么树，温度乱七八糟这些图转换成颜色的图，然后再加上说这个图里面的红色代表什么，绿色代表什么？黄色代表什么，然后让这个只在多模态，只在可见光里面学过的这个知识的模型去分析，就能分析出。对，所以说白了就是D说白就是一个标准的多模态的模型，能够用刚才这种方法分析这种多光谱的遥感数据了。对，然后别的就没啥了。

这有一个，我觉得最后这个挺有意思的。最后有两个信号，一个是AMD，我觉得AMD还是能还能继续还能继续涨，这个AMD的机会还是很大的。另外一个plantier还是很有大机会的。

然后最后一个就是分享这样一个叫AI engineer pass。这哥们儿发布了一个发表了一个什么呢？就是你作为一个初学的人，或者一个比较初级的工程师，你如果想学AI工程师的这方面的技能的话，应该按什么路径？我觉得这个还是写挺对的。

一个是你看先学掉大模型的API，再学点这个贴瓷的工程量，学点工具使用finding，稍微学一学就行了，不用太想。然后怎么去把外部的数据召回回来，就是做rag的一些东西，做rag然后进一步学一下agented rag，然后你可以做一些比较复杂的agent了，这个包括一些多agent，包括这个A2AACP这些东西。如果你想继续去全栈学的话，你就可以学一点这个info就是底层的模型的云服务，然后模型路由这些东西，安全这样的一些东西。如果想学的更前一点，找一点什么机器人，computer use，CLIA这些方面的点。所以我觉得如果大家有想去学的话，可以参考这张图来去定位一下自己，千万别一步跳的太猛了，对吧？你前面学的这些东西已经能干很多事情。

好，今天正好十点，然后大家有没有什么问题？今天就讲这些了，这些今天大恰好没来，我一直在哔哔哔，不知道这个大这个是不是会显得比较无聊。大家有什么问题没有？不知大家是不是都过国庆，还说好多人还没回来。我感觉好像好多人还在出去玩了很久，本来再多请两天假，应该可以一直玩到很久。我这个好像漏出来了，其对这个他看。

又没了。不知道为什么。对，sorry。这个东西你看其实就是它里面会有大量的这种生成的短的视频，几秒钟的视频。因为缩量2，它能生成大概20秒。然后这种东西看也是个假的，这个什么好像是一个采访的这么个东西。然后刷刷你看大量就全都是这个塑料生成的这样一个内容。然后内容我看过几个，还行，就是能看懂。对，sorry，有邀请码，然后是那个pro用户，所以也所以可以直接用，我看能不能发邀请码。

还得改改地址。

我已经改了。大家有啥问题没有啥说了。对，没改过去。

还没有人切好。

这个都是可以直接check，就是这个。

出来了。对，so 2。对，你看基本上里面的内容都是这种的。

这都啥东西？真的是网速太慢了，这肯定不是单纯的我的网速的问题。看90年代的什么东西是。哪儿能搞邀请码，我看看。大家可以自己去考这个屏幕，这个邀请码。大家自己截，我有四个，那就是我有四个邀请码，大家自己截，能看到吧？EX9P6S。

对，好，今天就差不多这些。然后大家有没有什么问题的话，那我们就下周再见好吧？好，行，那先这样，拜拜大家晚安。

